Industrial Robot Copilot - Committee Runbook

Project Summary

Industrial Robot Copilot is a multimodal safety and navigation assistant for industrial robots. It combines RGB, thermal, radar, VLM scene understanding, open-vocabulary object localization, risk reasoning, and robot path planning in a Gradio dashboard.

The demo shows a robot-perspective view with:

- detected humans and industrial objects
- risk level and action recommendation
- desired ground-plane path
- speed-coded path segments
- stop point behavior when the robot must halt
- Qwen or heuristic VLM scene reasoning
- locate-anything.cpp bounding boxes

Main Demo Order

Run in this order on the AMD cloud machine:

1. Check or repair the Qwen/vLLM environment.
2. Start Qwen VLM server.
3. Start locate-anything.cpp HTTP wrapper.
4. Run benchmark.
5. Build or resume the cached stream output.
6. Start the Gradio demo server.

Terminal 1 - Fix And Start Qwen

From the project root:

```
cd /workspace/AMD_Hackathon
./scripts/fix_amd_vllm_env.sh
./scripts/start_qwen_vllm.sh
```

Expected Qwen endpoint:

```
http://localhost:8000/v1
```

Health check:

```
curl http://localhost:8000/v1/models
```

If Qwen is still unstable, the app can run with:

```
VLM_BACKEND=heuristic
```

Terminal 2 - Start locate-anything.cpp Wrapper

Start the wrapper around the locate-anything.cpp CLI:

```
cd /workspace/AMD_Hackathon
python scripts/locate_anything_cpp_server.py \
  --bin /workspace/locate-anything.cpp/build/examples/cli/locate-anything-cli \
  --model /workspace/locate-anything.cpp/models/locate-anything-q8_0.gguf \
  --mode fast \
  --host 127.0.0.1 \
  --port 8188
```

Health check:

```
curl http://127.0.0.1:8188/health
```

The wrapper caches repeated detections by model, mode, image path, image mtime, and prompt. This makes repeated app and cache runs faster.

Terminal 3 - Benchmark First

Run the benchmark before caching:

```
cd /workspace/AMD_Hackathon
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/benchmark_pipeline.py --frame-count 10
```

Outputs:

```
outputs/benchmark_pipeline.json
outputs/benchmark_pipeline.csv
```

The benchmark reports:

- `locator_cold_seconds`: locate-anything.cpp recomputed for the frame
- `locator_cached_seconds`: repeated wrapper call using cache when available
- `pipeline_seconds`: full app pipeline using normal cache behavior
- `pipeline_non_locator_seconds`: estimated non-locator pipeline time
- `estimated_cold_e2e_seconds`: cold locator plus non-locator pipeline estimate

Latest measured sample from AMD run:

```
Average locator latency: 4.837s
Average pipeline latency: 2.823s
```

Interpretation:

- The locator is the current bottleneck on cold frames.

- Pipeline time can be lower than locator time when the wrapper cache is warm.
- For jury numbers, report both cold locator and warm/cached demo latency.

To measure a fully warm run:

```
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/benchmark_pipeline.py --frame-count 10 --use-existing-locator-cache
```

Monitor AMD GPU Use

In another terminal:

```
watch -n 1 rocm-smi
```

For one-time detailed info:

```
rocm-smi
```

For locate-anything.cpp build validation:

```
grep -R "GGML_HIP" /workspace/locate-anything.cpp/build/CMakeCache.txt
grep -R "LA_GGML" /workspace/locate-anything.cpp/build/CMakeCache.txt
ldd /workspace/locate-anything.cpp/build/examples/cli/locate-anything-cli | grep -iE "hip|ro
```

Expected HIP-related build flags include:

```
GGML_HIP:BOOL=ON
GGML_HIP_GRAPHS:BOOL=ON
GGML_HIP_MMQ_MFMA:BOOL=ON
```

If `rocm-smi` shows high VRAM but `GPU%` stays near zero during detection, the binary may still be mostly CPU-bound or only loading memory without meaningful GPU compute.

Build Or Resume Cached Stream

Build the Qwen plus locator cache:

```
cd /workspace/AMD_Hackathon
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/cache_qwen_outputs.py --frame-count 621
```

The cache script saves after every frame and resumes automatically. If it stops, run the same command again and it will skip completed frames.

To recompute everything:

```
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/cache_qwen_outputs.py --frame-count 621 --force
```

Cache file:

```
cache/qwen_stream.json
```

Cached records include:

- Qwen scene output
- risk/action/mode/target speed
- navigation desired path and floor region
- locate-anything.cpp bounding boxes
- path and floor source

Start Gradio Demo

Start the full demo with public sharing:

```
cd /workspace/AMD_Hackathon
DEMO_FRAME_COUNT=621 \
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
GRADIO_SHARE=1 \
./start.sh
```

Expected output:

```
Running on local URL: http://0.0.0.0:7860
Running on public URL: https://...gradio.live
```

If Qwen is unavailable, run the stable fallback:

```
DEMO_FRAME_COUNT=621 \
VLM_BACKEND=heuristic \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
GRADIO_SHARE=1 \
./start.sh
```

If locate-anything.cpp is unavailable, run the safest fallback:

```
DEMO_FRAME_COUNT=621 \
VLM_BACKEND=heuristic \
LOCATOR_BACKEND=labels \
```

```
GRADIO_SHARE=1 \
./start.sh
```

What The Jury Should Notice

- Multimodal safety reasoning: RGB, thermal, radar, VLM, and locator outputs are fused into one robot command.
- Robot path is grounded to the floor region instead of floating over the image.
- Path color encodes target speed.
- STOP actions render as a point from the robot perspective, not a misleading end line.
- Qwen can provide scene and path estimates; local heuristics keep the demo robust if the VLM endpoint fails.
- locate-anything.cpp boxes are shown in the stream and in the detailed frame pane.
- Caching makes the long stream practical for a live demo while retaining model output per frame.

Common Troubleshooting

vLLM import error for huggingface-hub

Run:

```
./scripts/fix_amd_vllm_env.sh
```

This pins compatible versions for `huggingface-hub`, `fastapi`, `starlette`, and `prometheus-fastapi-instrumentator`.

vLLM HTTP 500 with `_IncludedRouter`

Run:

```
./scripts/fix_amd_vllm_env.sh
./scripts/start_qwen_vllm.sh
```

Cache says no cached output for this frame

Run or resume:

```
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/cache_qwen_outputs.py --frame-count 621
```

Only first frames are shown

Start with:

```
DEMO_FRAME_COUNT=621 ./start.sh
```

locate-anything.cpp is slow

Check:

```
watch -n 1 rocm-smi
grep -R "GGML_HIP" /workspace/locate-anything.cpp/build/CMakeCache.txt
```

Use wrapper cache for demo runs:

```
LOCATOR_BACKEND=locate_anything_cpp_http
```

For fastest uncached throughput, the next engineering step is replacing the CLI wrapper with a native long-running C/C++ server that keeps the GGUF model loaded.